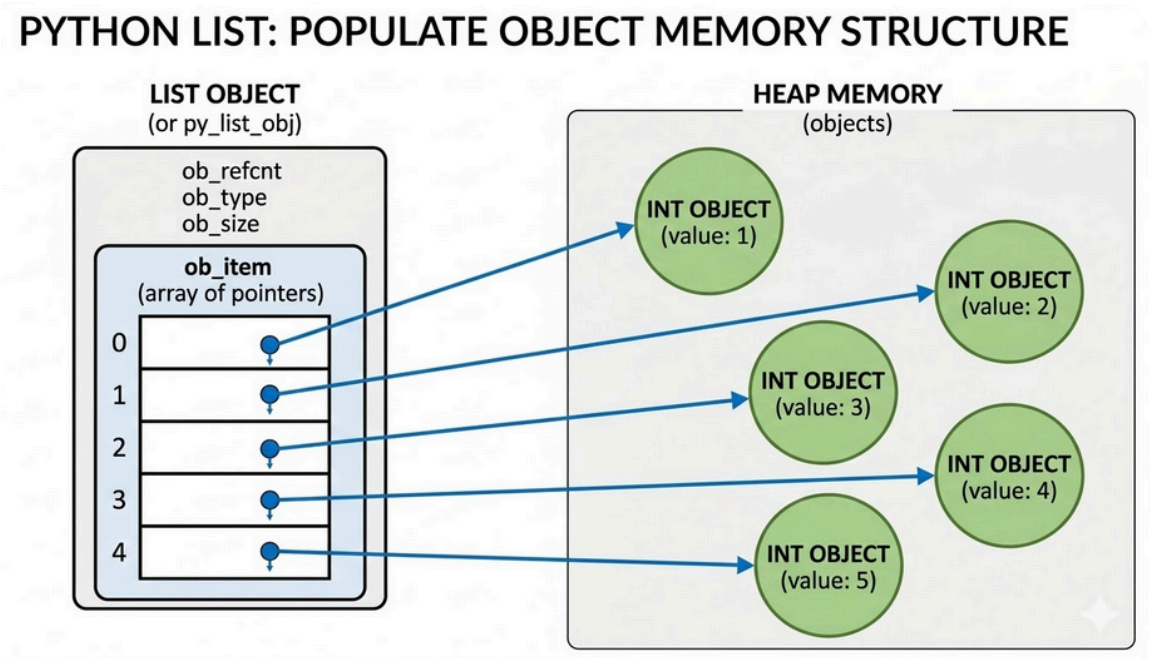
# 1️⃣ Parte izquierda: el objeto `list`

***sum_numbers = [1, 2, 3, 4, 5]***




El rectángulo de la izquierda es la estructura interna de la lista en **CPython**.

```
LIST OBJECT (PyListObject)
```

Campos importantes:

```
ob_refcnt   → contador de referencias
ob_type     → tipo del objeto (list)
ob_size     → tamaño de la lista
ob_item     → array de punteros
```

El campo clave es:

```
ob_item
```

Este es **un array de punteros (direcciones de memoria)**.

Ejemplo si tienes:

```
a = [1,2,3,4,5]
```

Internamente sería algo así:

```
ob_item

index | pointer
0     | → int(1)
1     | → int(2)
2     | → int(3)
3     | → int(4)
4     | → int(5)
```

Pero esos **enteros no están dentro de la lista**.

---

# 2 Parte derecha: el HEAP

En el heap están los **objetos reales**:

```
INT OBJECT (value:1)
INT OBJECT (value:2)
INT OBJECT (value:3)
INT OBJECT (value:4)
INT OBJECT (value:5)
```

La lista solo guarda **punteros a esos objetos**.

Por eso Python dice:

> Todo es un objeto.

---

# 3 Qué significa el array de punteros

La lista contiene algo parecido a esto en memoria:

```
ob_item = [ptr1, ptr2, ptr3, ptr4, ptr5]
```

donde:

```
ptr1 → dirección del objeto 1
ptr2 → dirección del objeto 2
ptr3 → dirección del objeto 3
...
```

---

# 4️⃣ Tu pregunta clave:

## ¿Cómo se calcula moverse de posición 1 a 2?

Aquí entra el concepto de **array en memoria contigua**.

Un array se recorre usando:

```
direccion = base + (indice * tamaño_del_elemento)
```

---

## En Python list

Los elementos del array **son punteros**, no los valores.

En una máquina de **64 bits**:

```
tamaño puntero = 8 bytes
```

---

### Supongamos

Dirección base del array:

```
ob_item = 1000
```

Entonces:

| índice | cálculo | dirección |
|---|---|---|
| 0 | 1000 + 0×8 | 1000 |
| 1 | 1000 + 1×8 | 1008 |
| 2 | 1000 + 2×8 | 1016 |

| | | |
|---|---|---|
| 3 | 1000 + 3×8 | 1024 |

---

### Entonces

Moverse de índice 1 → 2 significa:

```
direccion += 8 bytes
```

porque cada posición guarda **un puntero**.

---

# 5 Visualización real

```
Heap (objetos)

5000 → int(1)
5010 → int(2)
5020 → int(3)
5030 → int(4)
5040 → int(5)
```

Array de punteros:

```
1000 → 5000
1008 → 5010
1016 → 5020
1024 → 5030
1032 → 5040
```

Entonces:

```
lista[1]
```

Python hace:

```
direccion = 1000 + 1*8
leer puntero → 5010
ir al heap → objeto int(2)
```

---

# 6 Por qué Python usa punteros

Porque una lista puede contener:

```
[1, "hola", [1,2], objeto, función]
```

Todos tienen **tamaños distintos**.

Entonces Python guarda **punteros** a objetos en heap.

---

# 7 Coste de acceder a un elemento

Acceder a

```
a[3]
```

es **O(1)** porque:

```
direccion = base + indice * tamaño
```

No necesita recorrer la lista.

---

# 8 Algo MUY importante

Por esto sucede esto:

```
a = [1,2,3]
b = a
```

No se copia la lista.

Solo se copia el **puntero al objeto list**.

---

# 9️⃣ Y por esto pasa esto

```
a = [1,2,3]
b = a

b.append(4)

print(a)
```

Resultado:

```
[1,2,3,4]
```

Porque:

```
a y b → mismo objeto list
```

---

# 🔟 Detalle avanzado (CPython real)

La estructura interna real es:

```
typedef struct {
    PyObject_VAR_HEAD
    PyObject **ob_item;
    Py_ssize_t allocated;
} PyListObject;
```

donde:

```
ob_item = puntero a array de PyObject*
```

---

Cuando se dice:

> **En una máquina de 64 bits el tamaño de un puntero es 8 bytes**

significa **cuántos bytes necesita la computadora para guardar una dirección de memoria**.

Vamos paso a paso.

---

# 1 Qué es un puntero

Un **puntero** es simplemente **una dirección de memoria**.

Ejemplo conceptual:

```
0x00007FF8A1C03000
```

Eso le dice al programa:

> "El objeto está guardado en esta posición de la RAM".

---

# 2 Qué significa “64 bits”

Un sistema de **64 bits** significa que las direcciones de memoria usan **64 bits**.

Un **bit** puede ser:

```
0 o 1
```

Entonces:

```
64 bits = 64 posiciones binarias
```

---

# 3 Conversión a bytes

Sabemos que:

```
1 byte = 8 bits
```

Entonces:

```
64 bits ÷ 8 = 8 bytes
```

Por eso:

```
puntero = 8 bytes
```

---

# 4 Comparación con 32 bits

| arquitectura | tamaño puntero |
|---|---|
| 32 bits | 4 bytes |
| 64 bits | 8 bytes |

Porque:

```
32 bits ÷ 8 = 4 bytes
64 bits ÷ 8 = 8 bytes
```

---

# 5 Cómo se ve una dirección real

En un sistema 64 bits:

```
0x00007FFE34A1B0F0
```

Ese número ocupa **8 bytes**.

---

# 6 Qué significa para una lista en Python

Cuando Python crea una lista:

```
a = [1,2,3,4]
```

La lista guarda **punteros a objetos**.

En memoria sería algo así:

```
ob_item

[ ptr ][ ptr ][ ptr ][ ptr ]
```

Cada `ptr` ocupa:

```
8 bytes
```

Entonces el array ocupa:

```
4 elementos × 8 bytes = 32 bytes
```

(Solo los punteros, no los objetos).

---

# 7 Cómo se calcula el acceso

Para acceder a un elemento Python usa:

```
direccion = base + indice * tamaño_elemento
```

Como el elemento es un puntero:

```
tamaño_elemento = 8 bytes
```

Ejemplo:

```
base = 1000
```

| índice | cálculo | dirección |
|---|---|---|
| 0 | 1000 + 0×8 | 1000 |
| 1 | 1000 + 1×8 | 1008 |
| 2 | 1000 + 2×8 | 1016 |

Moverse de índice **1** → **2**:

```
1008 → 1016
```

Se suman **8 bytes**.

---

# 8 Qué pasa en memoria

Lista:

```
ob_item
1000 → ptr
1008 → ptr
1016 → ptr
1024 → ptr
```

Heap:

```
5000 → int(1)
5010 → int(2)
5020 → int(3)
```

Entonces:

```
lista[1]
```

Python hace:

```
1000 + 1*8 = 1008
leer puntero
ir al heap
```

---

# 9 Cómo comprobarlo en Python

Puedes ver el tamaño de un puntero así:

```
import struct
print(struct.calcsize("P"))
```

Resultado en sistemas 64 bits:

---

# 🔟 Algo importante

Aunque el puntero sea de **8 bytes**, los objetos Python son mucho más grandes.

Por ejemplo:

```
int en Python ≈ 28 bytes
```

Porque Python guarda:

- tipo del objeto
- reference count
- valor
- metadata

---